Prisma Usage

This document describes how Prisma is used throughout the backend for database operations.

Prisma Client Setup

The Prisma Client is imported and instantiated in `backend/src/server.js`:

```
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
```

Common Query Patterns

Finding Records

```
// Find one by unique field
const user = await prisma.user.findUnique({
  where: { email: 'admin@example.com' }
});

// Find first matching record
```

```
const settings = await prisma.systemSettings.findFirst();

// Find many with filters
const openTickets = await prisma.ticket.findMany({
  where: { status: 'Open' },
  orderBy: { createdAt: 'desc' }
});
```

Creating Records

```
const ticket = await prisma.ticket.create({
  data: {
    subject: 'Printer not working',
    description: '...',
    status: 'Open',
    priority: 'Medium',
    requesterEmail: 'user@example.com',
    ticketNumber: await generateTicketNumber()
  }
});
```

Updating Records

```
const updated = await prisma.ticket.update({
  where: { id: ticketId },
  data: {
    status: 'Resolved',
    resolvedAt: new Date()
  }
});
```

Deleting Records

```
await prisma.ticket.delete({
  where: { id: ticketId }
});
```

Relations

Including Related Data

```
const ticket = await prisma.ticket.findUnique({
  where: { id: ticketId },
  include: {
    assignedTo: true,
    department: true,
    comments: {
      include: { user: true },
      orderBy: { createdAt: 'asc' }
    },
    attachments: true,
    history: {
      orderBy: { createdAt: 'desc' }
    }
  }
});
```

Creating Related Records

```
const comment = await prisma.comment.create({
```

```
  data: {
    content: 'Working on this issue',
    isInternal: false,
    ticketId: ticketId,
    userId: currentUser.id
  }
});
```

Transactions

For operations that must succeed or fail together:

```
const result = await prisma.$transaction(async (tx) => {
  const ticket = await tx.ticket.create({ data: ticketData });
  await tx.ticketHistory.create({
    data: {
      ticketId: ticket.id,
      field: 'status',
      oldValue: null,
      newValue: 'Open',
      changedBy: currentUser.id
    }
  });
  return ticket;
});
```

Raw Queries

When Prisma's query builder is insufficient:

```
const results = await prisma.$queryRaw`
  SELECT
    d.name,
    COUNT(t.id) as ticket_count
  FROM Department d
  LEFT JOIN Ticket t ON t.departmentId = d.id
  GROUP BY d.id, d.name
  ORDER BY ticket_count DESC
`;
```

Error Handling

Prisma throws specific error codes:

Code	Meaning
`P2002`	Unique constraint violation
`P2025`	Record not found
`P2003`	Foreign key constraint violation
`P2014`	Required relation missing

```
try {
  await prisma.user.create({ data: userData });
} catch (error) {
  if (error.code === 'P2002') {
    // Handle duplicate email/username
```

```
  }
}
```

Connection Management

Prisma manages connection pooling automatically. For serverless environments, consider using `@prisma/extension-pulse` or connection management tools.

Performance Tips

1. **Use `include` selectively** — only fetch related data when needed.
2. **Use `select` for large tables** — avoid fetching unnecessary columns.
3. **Add indexes** for frequently filtered fields.
4. **Use pagination** for large result sets.
5. **Use `$transaction`** for batch operations.

Previous
« Seed Data

Next
API Overview »

Docs

Getting Started

Architecture

API Reference

Community

GitHub

More

Deployment Guide

Troubleshooting

Copyright © 2026 KUCCPS. Built with Docusaurus.